

CLOUD NATIVE
COMPUTING FOUNDATION

Kelpflux

Elastic Slurm scheduling on Kubernetes
for shared GPU AI workloads.

Agenda

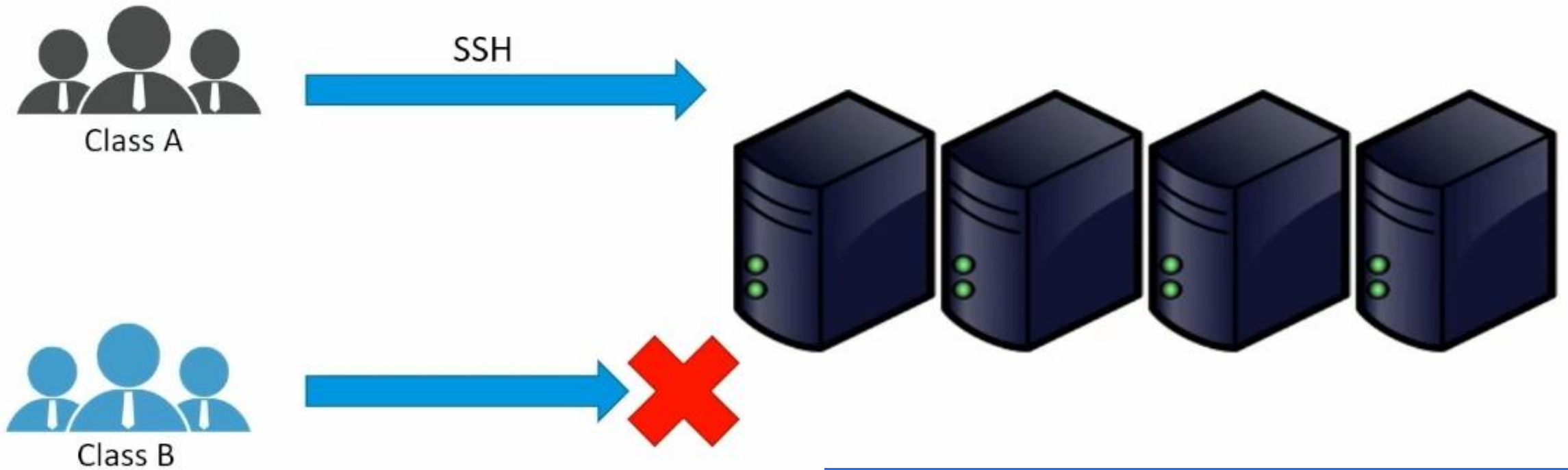
-  **1 Introduction**
-  **2 System Architecture**
-  **3 Demo & Evaluation**
-  **4 References**



01. Introduction

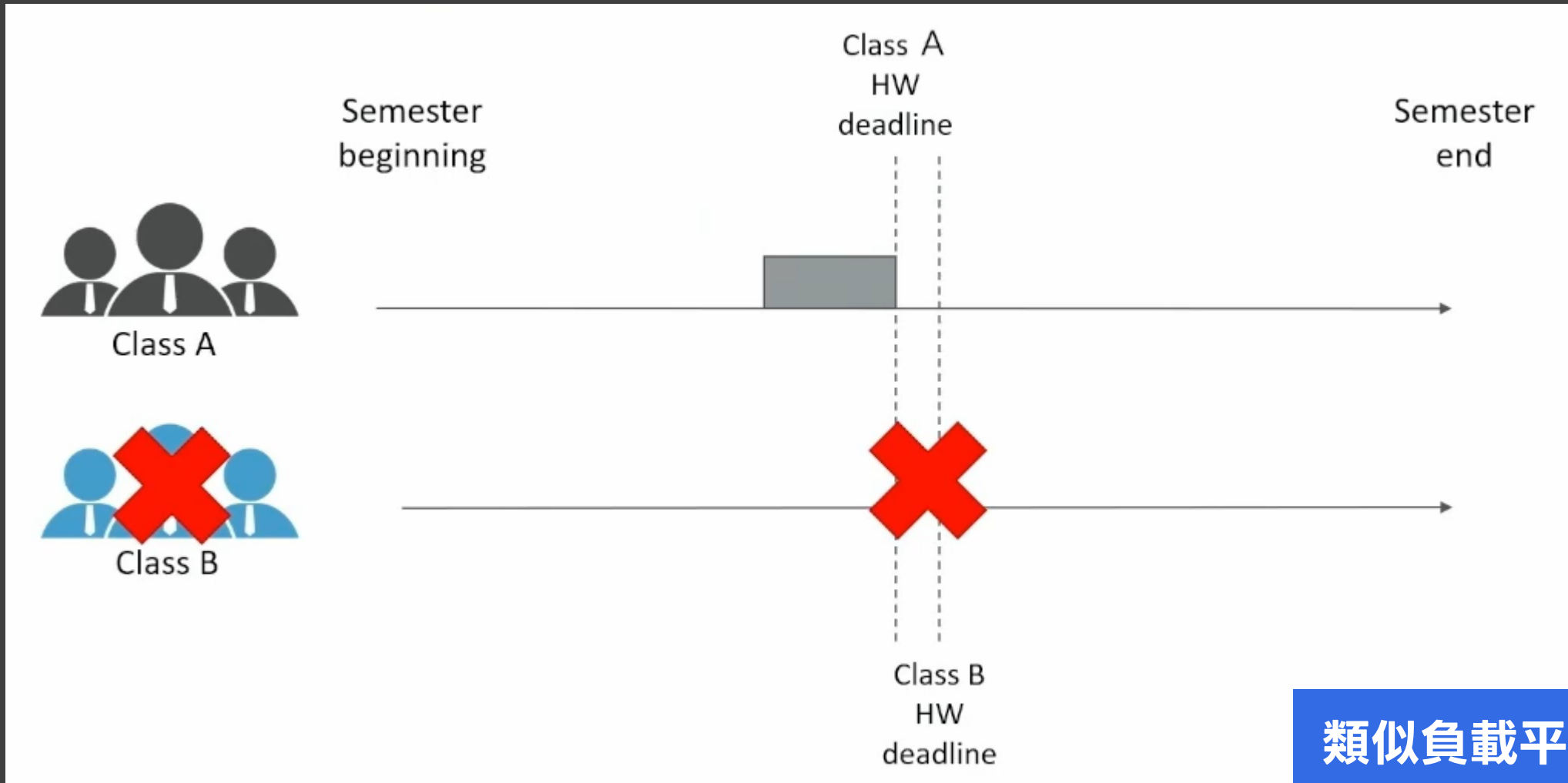
專題簡介

Example Scenarios



有沒有辦法多台機器給多個人使用？

Example Scenarios



類似負載平衡的概念

Slurm

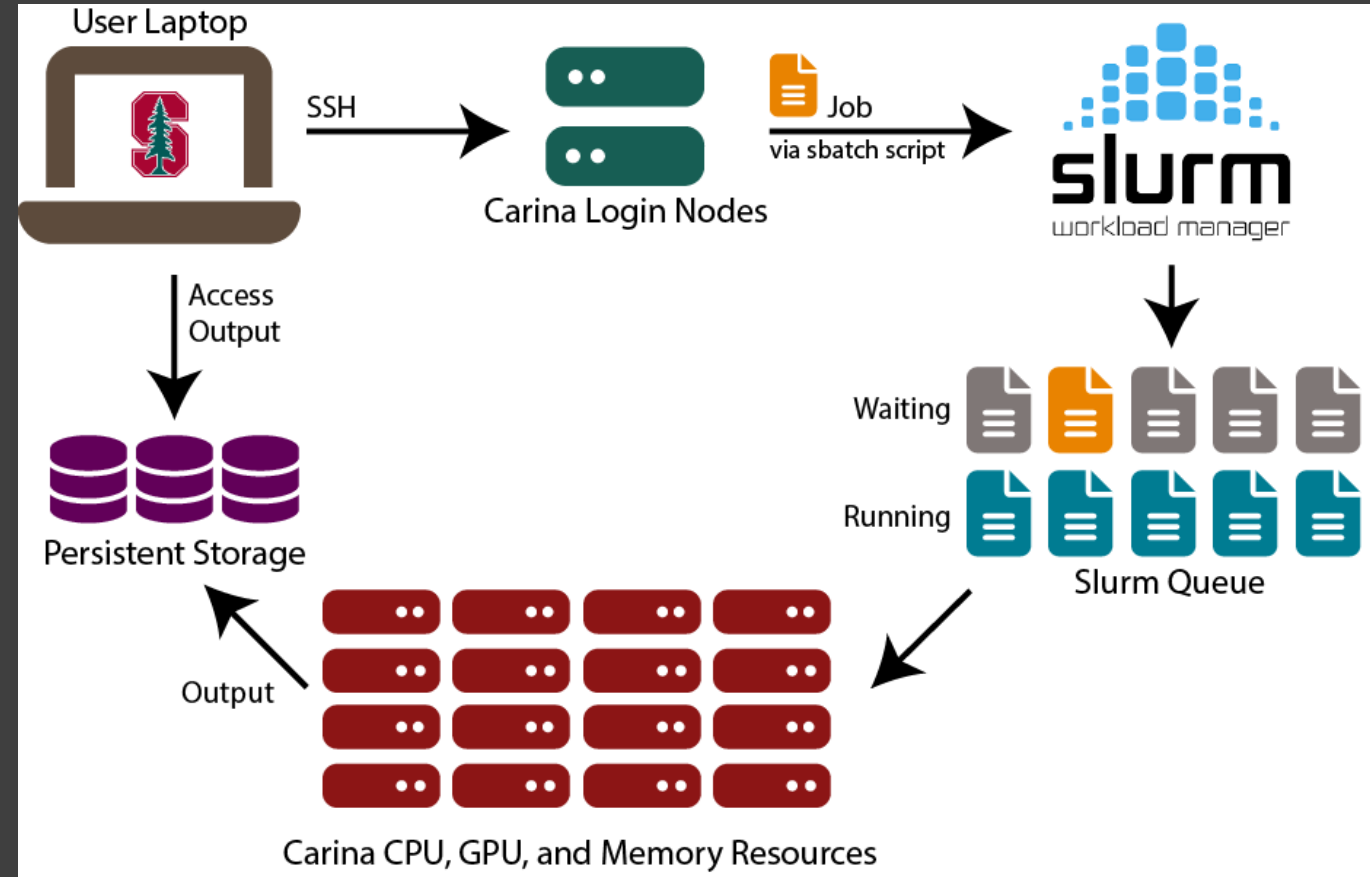
What is Slurm ?

- Slurm = Simple Linux Utility for Resource Management
- An open source system
 - 管理工作：資料分配、排程、執行



Roles in Slurm

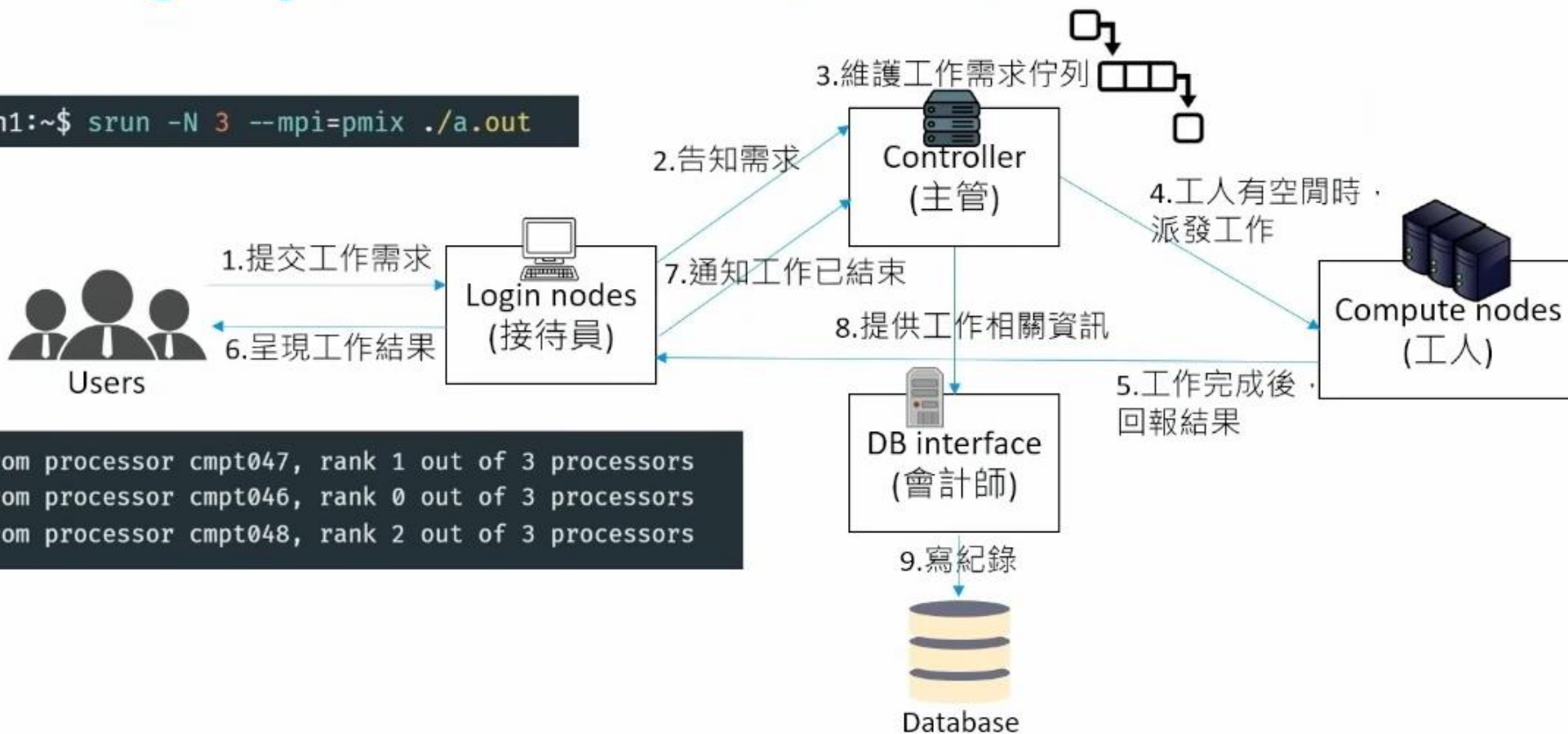
- Login nodes = 接待員/客服
- Controller = 主管
- Compute nodes = 工人
- DB interface = 會計師



Running a job in Slurm (srun)

Running a job in Slurm (srun)

```
bjhuang@login1:~$ srun -N 3 --mpi=pmix ./a.out
```



```
Hello world from processor cmpt047, rank 1 out of 3 processors  
Hello world from processor cmpt046, rank 0 out of 3 processors  
Hello world from processor cmpt048, rank 2 out of 3 processors
```

Kubernetes

What is Kubernetes ?

- Kubernetes (K8s) = Containers Orchestration Tool
- An open source system
 - 管理容器：啟動、運算資源、排程、自動部署
 - 大公司架設服務很愛用

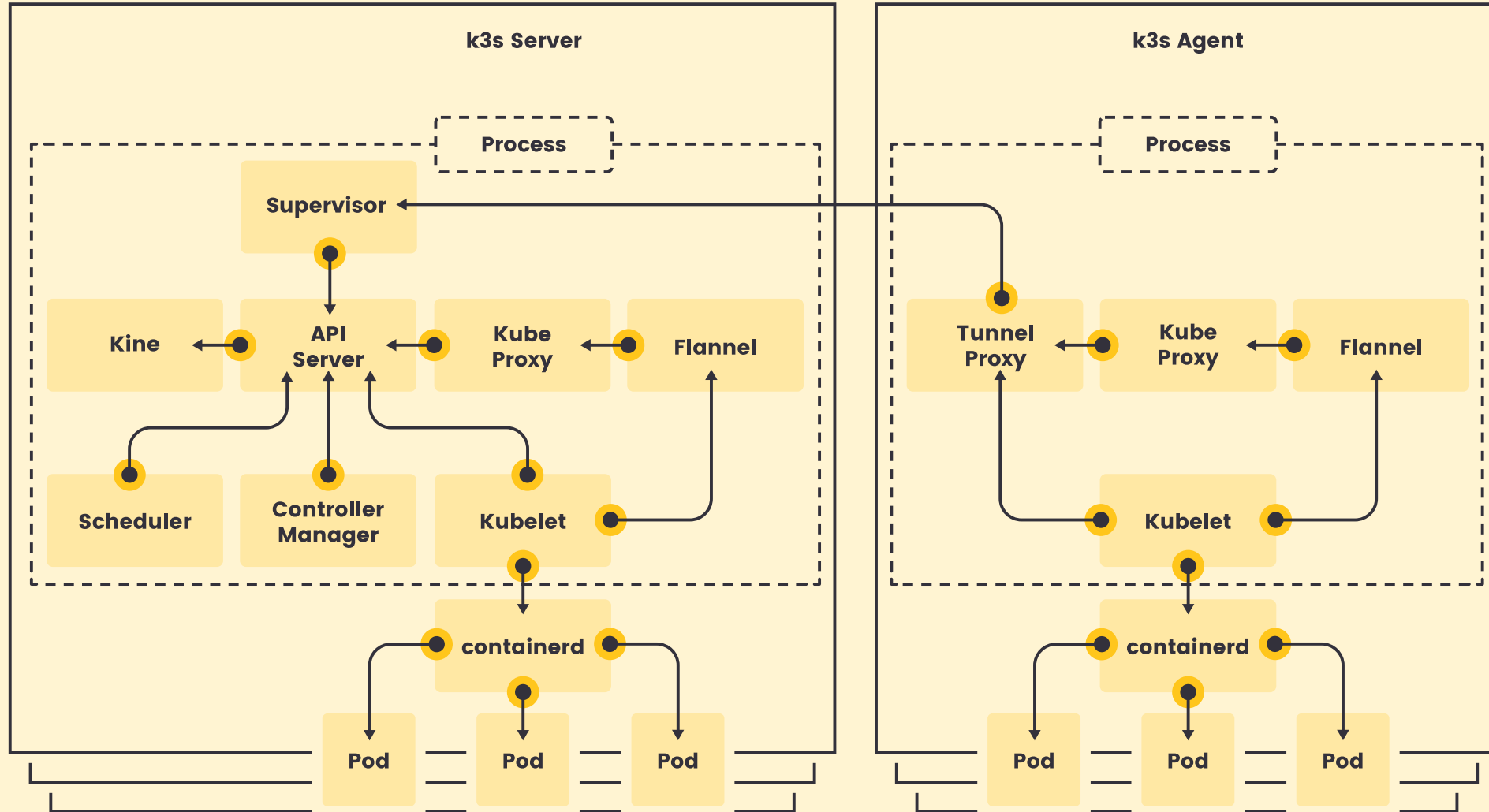


Kubernetes Distribution

- 本來是用 kind，但硬體支援度有極限，改成使用 K3s
- 建立在 Kubernetes 標準安裝之上的一個輕量發行版
- 由 Rancher Labs 開發，通過 CNCF 認證。



K3s



Background

Motivation

- Kubernetes 很擅長資源編排與彈性伸縮，但對 HPC / AI 訓練工作負載的語意支援有限。
 - Slurm 很擅長批次排程、資源配額與叢集治理，但傳統部署通常偏靜態，對雲端環境的節點生命週期不夠友善。
- ⇒ 打造一個讓多位使用者共用的批次 AI 工作平台



Benefits

- 提升資源利用率
 - Slurm 可以透過將工作排程到 K8s 叢集中的可用資源
- 簡化的工作負載管理
 - 一律透過 Slurm 介面，用於管理 ML 工作負載
- 提升可擴展性
 - Slurm 可以透過將機器學習工作負載排程到多個 K8s 叢集來擴展

Slurm on Kubernetes

- Slurm 決定誰用資源、什麼時候用、用多少
 - 計算資源排程問題
- Kubernetes 決定資源怎麼存在、怎麼啟動、怎麼管理
 - 動態提供 Slurm 需要的運算節點
 - 當硬體調整的時候不用重新調整 Slurm 設定
 - 依賴版本管理、網路設定

Our Feature !



- 擴容、擴縮、低功耗、節點壞掉重啟
- 客製化排程演算法、運算資源分配
- 支援多種 Job
- 近年來企業很重視安全，可以裝個龍蝦



Reliability



Performance



Lifecycle Unity



Observability



02. System Architecture

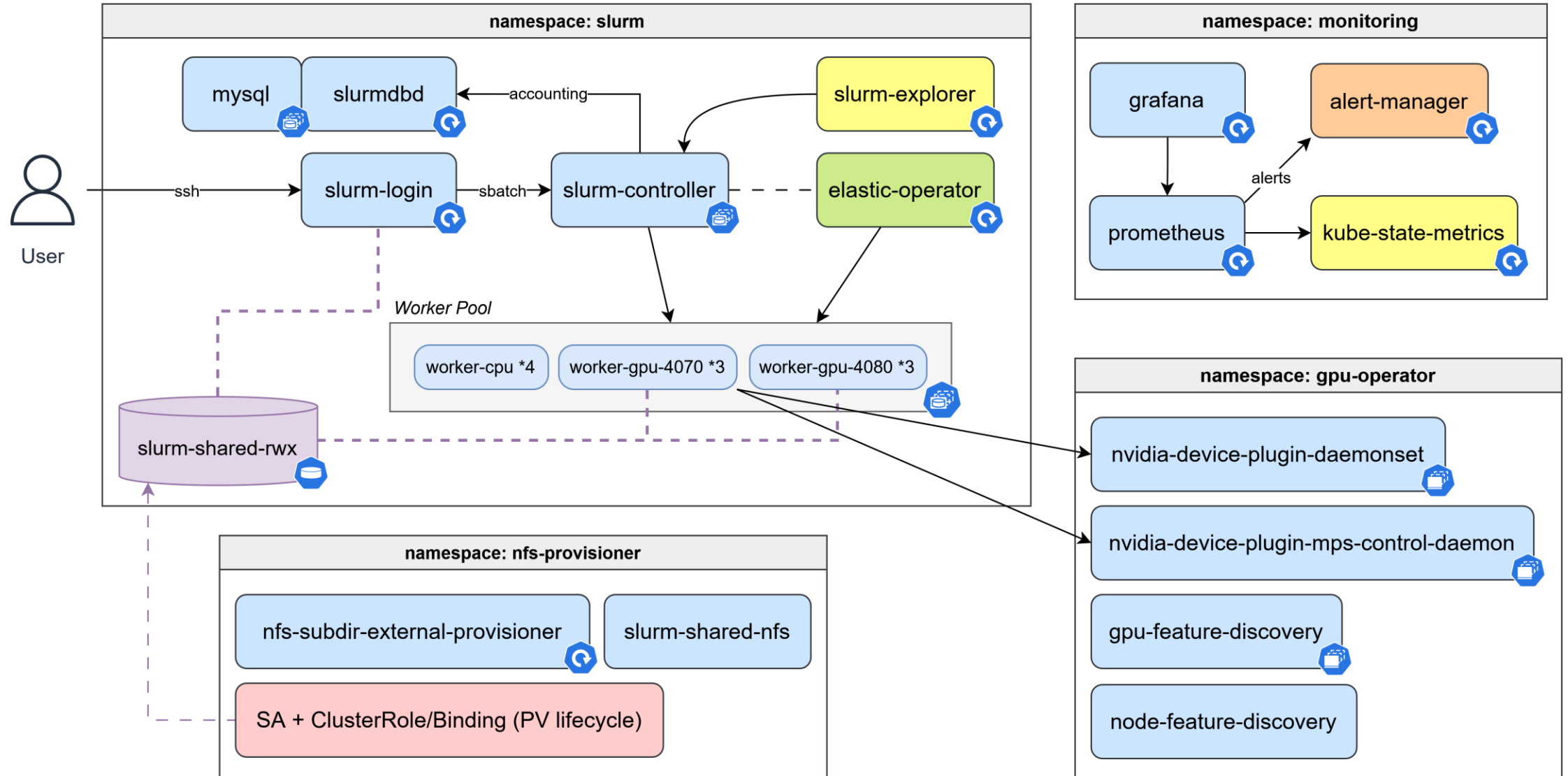
系統設計與架構

Development Environment



- Ubuntu 24.04 + K3s v1.34 + RTX 4070 (lab)
- Operator : Python 3.11-slim + kubectl CLI
- Nvidia GPU Operator

K8s (K3s) Cluster



Slurm GRES

- Slurm 的通用資源管理機制，讓 Slurm 能夠追蹤和排程 CPU/Memory 以外的資源 (e.g. GPU)
- 知道每個 node 上有哪些 GPU (型號、數量)
- 讓使用者提交 job 時指定需要的 GPU 類型和數量
- 確保 GPU 資源不會被過度分配

GPU Resources

Multi-Process Service (MPS)

- 在 GPU 上創建多個 context，將 GPU 的資源分配給多個 process
- 每個進程擁有一個獨立的 context，可以獨立執行其計算任務
- 提高多容器共享 GPU 的 QoS
- 不管 VRAM、記憶體頻寬、L2 Cache、編解碼器、Copy Engines
 - export `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0,3072` (slot_0 3072 MB)

NVIDIA GPU MPS



請求 + constraint	RTX 4070 分配 SM	RTX 4080 分配 SM	適合工作
<code>--gres=mps:50 --constraint=gpu-rtx4070</code>	~24 SM (50%)	—	中型推論 (7B LLM serving)
<code>--gres=mps:25 --constraint=gpu-rtx4070</code>	~12 SM (25%)	—	小型推論 (image classifier)
<code>--gres=gpu:rtx4070:1</code>	48 SM (整卡)	—	訓練 (≤12 GB VRAM)
<code>--gres=gpu:rtx4080:1</code>	—	76 SM (整卡)	訓練 (≤16 GB VRAM)
<code>-N 2 --gres=gpu:1</code>	48 SM	76 SM	2-GPU DDP

Job Migration

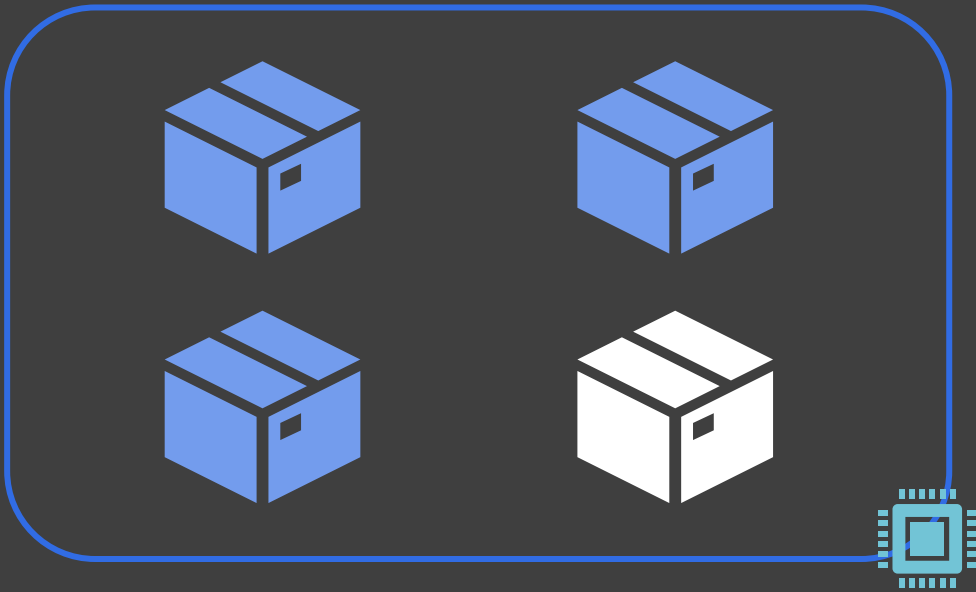
Job A



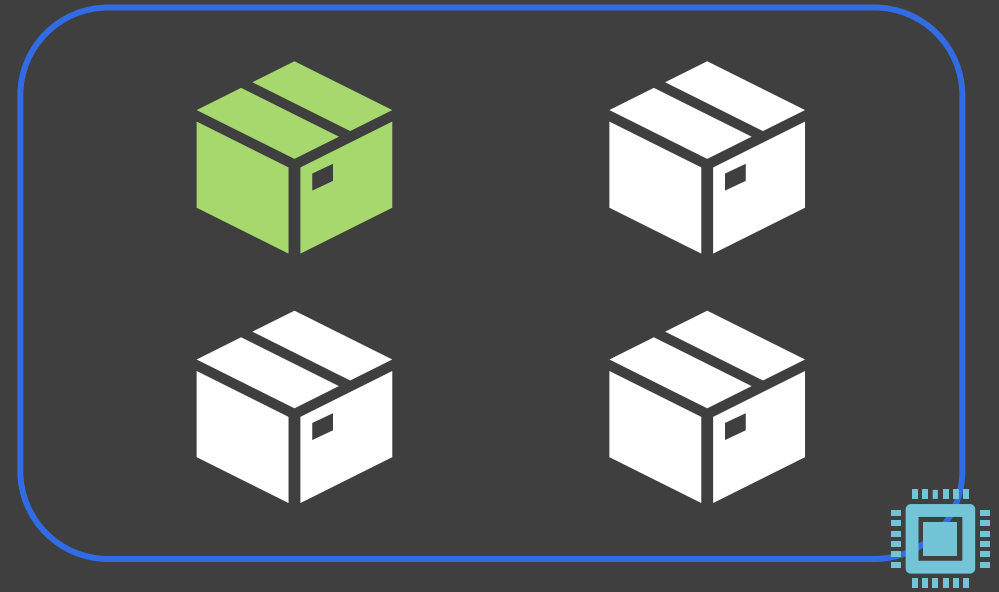
Job B



Job C

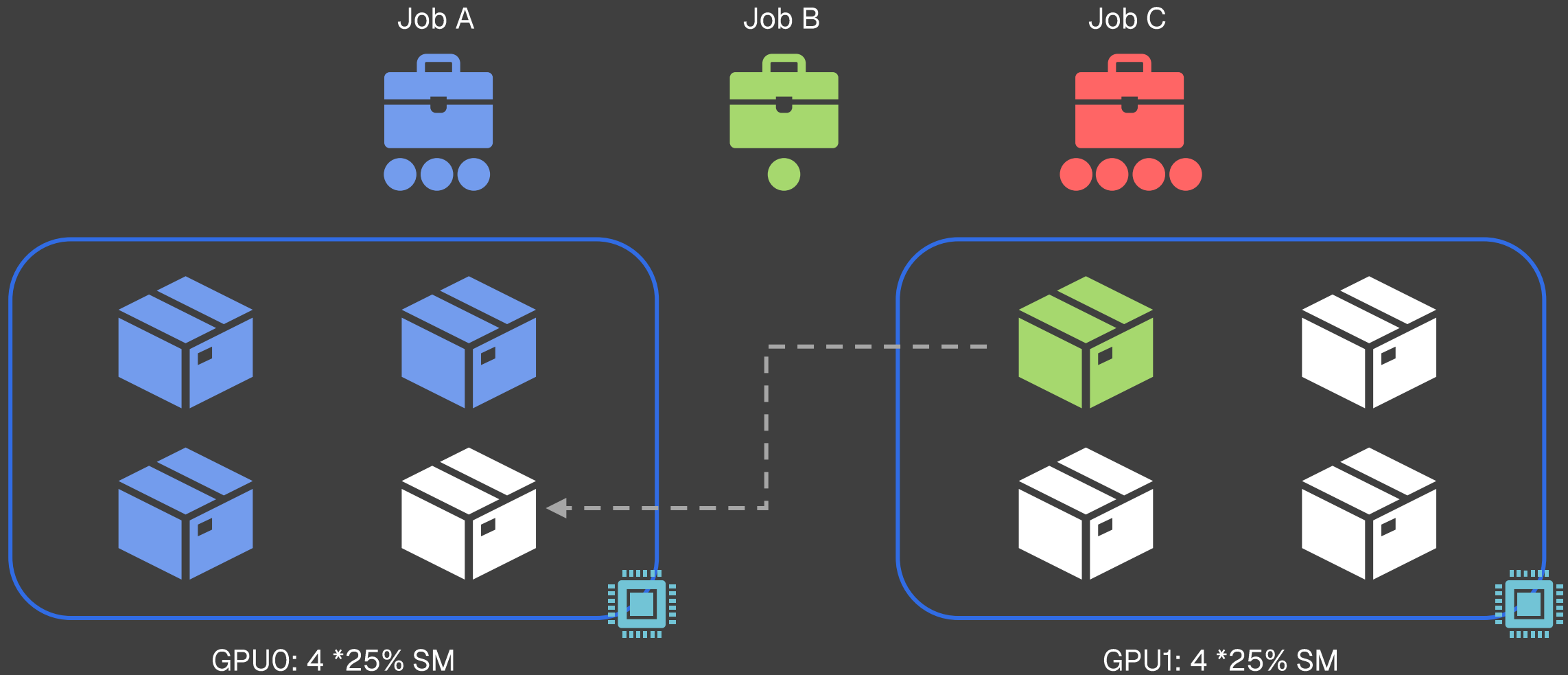


GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Migration



Job Migration

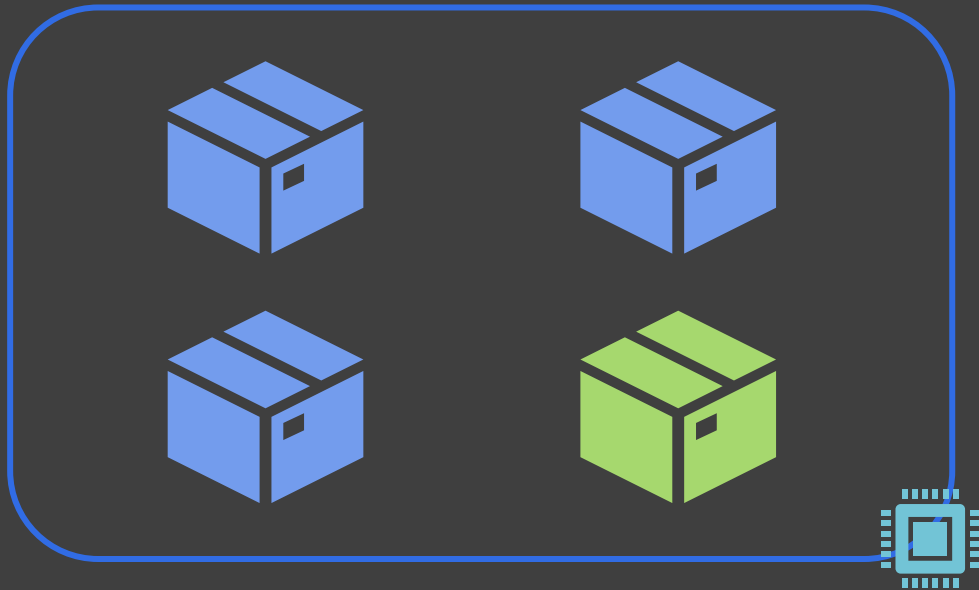
Job A



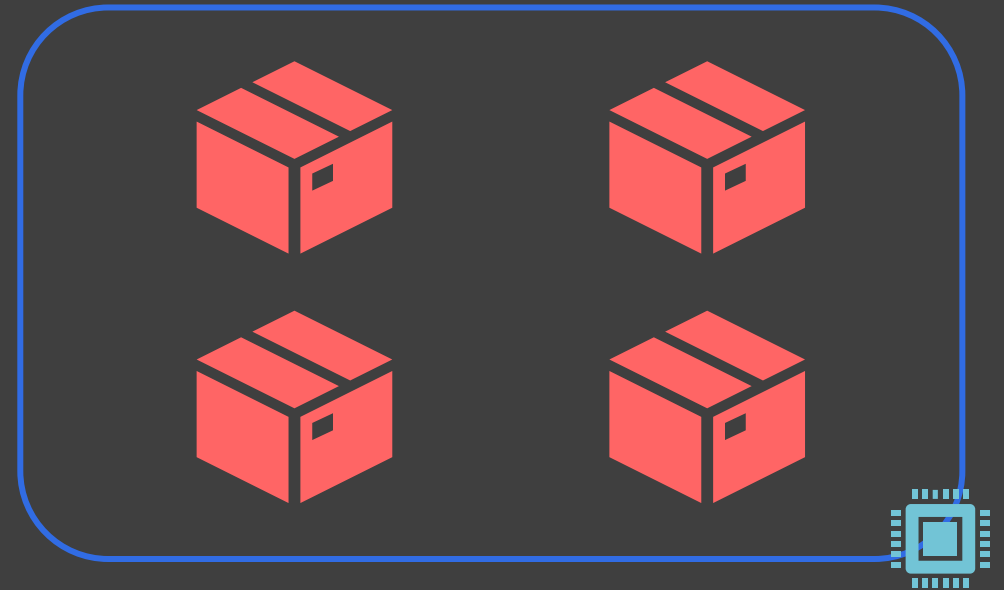
Job B



Job C



GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Scheduling / Migration

Slurm Scheduling Strategy (1/2)

- Bin-packing：先塞滿第一張卡，再用第二張卡
- 預設情況下，Slurm 為了散熱和負載平衡，傾向於把 Job 分散到不同的節點。

```
slurm:
  config: |
    SelectType=select/cons_tres
    # 重點：CR_Core_Memory 配合 CR_Pack_Nodes
    # 這會讓 Slurm 優先把任務填入同一個資源塊
    SelectTypeParameters=CR_Core_Memory,CR_Pack_Nodes
```

Slurm Scheduling Strategy (2/2)

- Preemption：需要定義不同的 Partition 優先級
 - HighPrio Partition: 給 Job C (重要大任務)，優先級 10。
 - LowPrio Partition: 給 Job B (小任務)，優先級 1。
- 當 Job C 提交到 HighPrio 時，Slurm 會自動把 GPU1 上的 Job B 踢掉，讓 Job C 執行
- 等到 GPU0 或 GPU1 有空位時，Job B 會自動重新啟動

```
slurm:
  config: |
    PreemptMode=REQUEUE           # 或者用 SUSPEND (如果 Job 支援)
    PreemptType=preempt/partition_prio
    JobRequeue=1
```

Scheduling Strategy Design

- 啟發式排程：改 Slurm 設定即可，快速有效
- 數學式排程：尋找最佳解
 - Analytical modeling + optimization
 - 深度強化學習 (DRL)

Analytical modeling + Optimization

- Score Function：給每個 (job, placement) 算分，挑最高的
- 係數 $\alpha, \beta, \gamma, \delta, \varepsilon$ 怎麼定？
 - 線上學習：把 $\alpha - \varepsilon$ 當可學習的權重，用 UCB1 等演算法線上更新。
 - 深度強化學習 DRL

Scheduler Summary (1/3)

$$\text{priority} = \alpha \times f_{mps_fit} + \beta \times f_{vram_fit} + \gamma \times f_{topology} \\ - \delta \times f_{fragmentation} + \epsilon \times f_{pred_runtime}$$

- job 需要的 MPS slot vs node 剩餘 slot 的配適度 → 讓小 job 填充空隙
- VRAM 需求 vs. node 等級 → 不浪費高規 GPU
- ~~拓撲屬性~~
- 接受後 cluster free-MPS 的 懲罰項 → 不製造碎片
- $\text{clamp}(1 - \text{pred_s} / 14400)$ → 短 job 優先

MPS Fit

- 衡量作業申請的 MPS slots 是否能有效填滿節點剩餘空間，鼓勵「剛好塞滿」的行為。
- 計算方式： $\text{mps_req} / \text{MPS_PER_NODE} (100)$
- 邏輯：如果作業沒有申請 MPS，則回傳 1.0（不懲罰）；若申請量超過節點上限則回傳 0.0

VRAM Fit

- 避免小顯存需求的作業佔用大顯存節點。
- 計算方式： $1 - (\text{fit_tier} - \text{req}) / \text{VRAM_MAX}$
- 定義 VRAM Tiers (2,4,8,...)，會尋找 $\geq$ 需求的最小 Tier。如果剛好符合則得 1.0，若跨級距則依比例扣分。

Runtime Evaluation

- 短作業優先 (SJF)
- 利用預測出的執行時間給予短作業加分。
- 執行時間越短 (small pred_seconds)，得分越接近 1.0
- 由 runtime predictor 計算預測時間

Runtime Predictor

- 預測服務的核心是一個基於 LightGBM 的回歸模型
- 模型從 Slurm 的歷史數據中提取特徵進行預測
 - 使用者資訊：user_name (假設不同使用者的作業模式具備穩定性)
 - 資源需求：gpu_count、mps_req、gpu_type
 - 時間特徵：hour_of_week (提交作業的時間點)
 - 歷史統計：user_freq、作業執行頻率與平均時長的對數
- 針對 log-runtime 的 MAE 進行優化目標函數

Default Weights

- $\alpha=0.40$ ：主導因子，鼓勵資源打包
- $\beta=0.2$ ：RAM 資源保護
- $\delta=0.20$ ：抑制碎片化
- $\varepsilon=0.30$ ：啟用預測器後的 SJF 權重

UCB1 Weight Tuner

- 計算每個權重組合的平均獎勵，並加上一個探索增益來決定下一次要測試的組合
- $\text{Reward} = -\text{jct_mean} / 3600$ 。跑完一整段作業歷史 (trace) 後，計算出的平均完成時間越短，獎勵越高
 - 從 Slurm 提取歷史數據來離線優化權重
 - 做完後 upgrade 到叢集中

UCB1 Bandit (RL)

- 選擇「目前看起來最好的」，還是去嘗試「那些還不熟悉的」？
- 為每一個選項計算一個分數，每次都選擇分數最高的那個
- 從 Slurm 提取歷史工作數據生成訓練用的 trace。
- 將選出的最佳權重手動更新至叢集屬性中

$$\text{UCB}_i = \underline{\bar{x}_i} + \sqrt{\frac{2 \ln N}{n_i}}$$

該選項過去的
平均獎勵

N 是總次數， n_i 是該
選項被選中的次數。

Scheduler Summary (1/2)

Deep RL

- 還沒上產線，因為目前 policy 還不可靠，要改善

UCB1 Weight tuner

- 每 300 秒查看工作歷程
- 用 bandit 自動調整 $(\alpha, \delta, \varepsilon)$ 參數 (online)
- Lua plugin 載入時覆蓋係數

Score + Slurm Settings

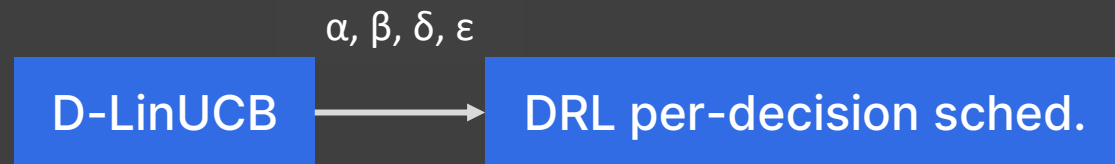
- 每次 sbatch 時計算分數並寫入 priority
- f_pred_runtime 呼叫 LightGBM 預測器服務
- Slurm backfill 排程器用 priority 來排序工作

Scheduler Summary (2/2)

- 公式候選參數： $\alpha = 0.10, \beta = 0.20, \delta = 0.05, \varepsilon = 0.30$
- 比較有效的設計是 runtime predictor + UCB1 weight-tuner。
- DRL 還在訓練及改善中

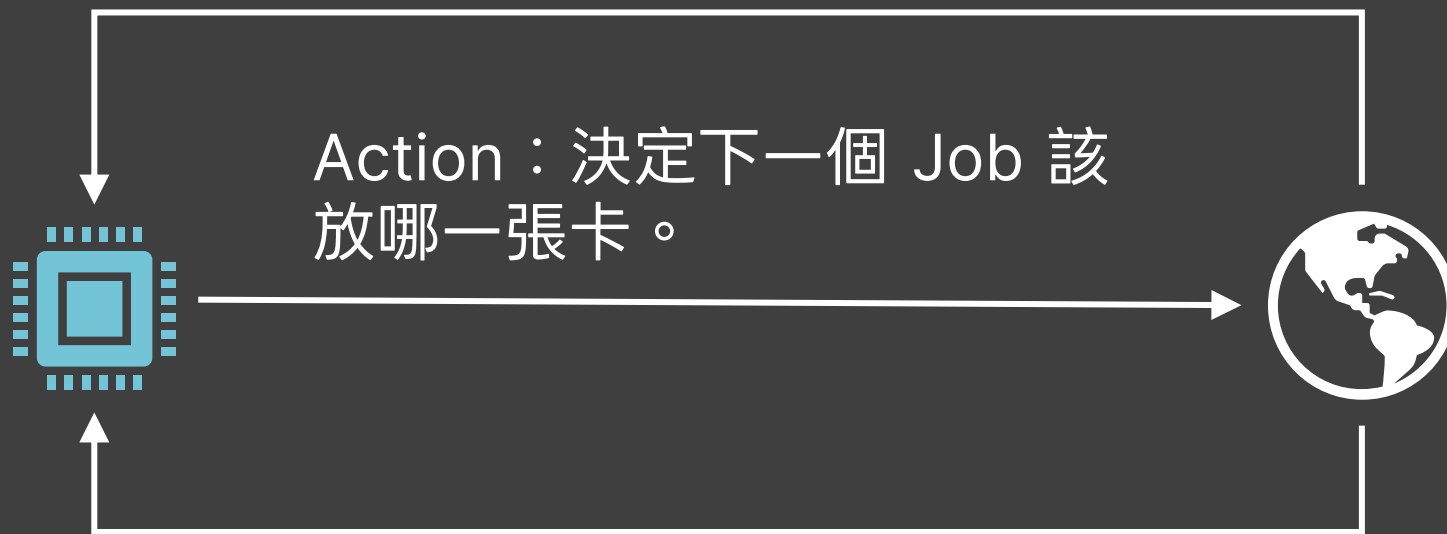
DRL Scheduler

- 直接在 live cluster 跑 online DRL 在小叢集不可行，因此設計：
 1. 模擬訓練：透過 PPO (近端策略優化) 拿到 policy
 2. 實戰微調：用 RLPD (Ball et al., 2023) 來做 fine-tune。
 3. 整合：外層 D-LinUCB (Russac, Y. et al.) + DRL 排程。



Deep Reinforcement Learning (DRL) ***

Reward：如果 Job 跑得快且資源沒浪費，給正分；如果發生 OOM 或排隊太久，給負分。



DQN, PPO,
DDPG,
DSAC*, ...

State：目前叢集所有 GPU 的剩餘資源、等待隊列中的 Job 屬性。

Scheduling Workflow

sbatch 時

job_submit.lua

- 算 score
- 寫入 job_desc.priority
- slurmctld 收到

slurmctld 排程時

Slurm backfill

- 取 queue 中
priority 最高 & feasible 的 job
- 呼叫 select/cons_tres
- 分配具體 node/GPU

DSAC

Paper Highlight

- SAC 探索能力強，但只看回饋期望值；Distributional RL 看整個回饋分布，但在連續控制與探索上不夠強。
- DSAC 嘗試把兩者合起來，讓 agent 同時學「期望表現」與「風險分布」
- 只看平均值會掩蓋風險！ 假設兩個 policy 的平均 reward 都是 100，但一個穩定拿 95~105，另一個有時拿 200、有時直接 crash，期望值可能接近，但實際決策偏好完全不同。這在白駕車、機器人控制、金融交易、醫療策略都很致命。

DSAC (Distributional Soft Actor-Critic)

- 第一，在 risk-neutral 場景中，用 return distribution 改善 value estimation，使 SAC 更穩、更準。
- 第二，在 risk-sensitive 場景中，讓 actor 不只最大化平均 reward，也能優化 CVaR、Wang distortion、CPW、mean-semideviation 這類風險指標。這是它比一般 SAC 更有研究價值的地方。

“Distributional” soft Bellman operator

- 把 SAC 的 soft Bellman operator 改成 distributional 版本

$$Q(s, a) = r + \gamma \mathbb{E}_{s', a'} [Q(s', a') - \alpha \log \pi(a' | s')]$$

$$Z^\pi(s, a) = R(s, a) + \sum_{t=1}^{\infty} \gamma^t (R(s_t, a_t) - \alpha \log \pi(a_t | s_t))$$

$$T_{DS}^\pi Z(s, a) = R(s, a) + \gamma (Z(s', a') - \alpha \log \pi(a' | s'))$$

Observability

Observability



雲端服務崛起



錯誤排查難度上升

Observability Aspects

Metrics



負責監控系統有什麼狀況，當要發生服務故障前可以透過設定閾值搭配告警提早得知。

Logs

當問題發生時，可以用來查看故障時正在執行哪些服務，以及產生的錯誤資訊。

Traces



在連續的時間維度上，透過 Trace 以及 Span 關聯，把空間給排列展示出來，並且有 Trace-Context 規範，能夠直觀的看到請求在分散式系統中經過所有服務的紀錄。



- 最受歡迎的開源監控與告警工具組，是 CNCF 的畢業專案
- 專為動態雲端環境設計，透過 HTTP 拉取 metrics，儲存於時間序列資料庫中，並使用 PromQL 進行數據查詢分析。
- 負責發 Alert 到 Slack/Telegram



- 能夠將各種資料做成監控儀表板，支援多種資料來源，以及提供豐富的面板選擇，協助我們將資料視覺化。
- Prometheus 預設的 GUI 功能不足，需要 Grafana 這種支援豐富圖表的工具幫助我們可視化 Metrics 資訊。



- 由 CNCF 孵化的開源專案，是 Cloud Native 的可觀測性框架
- 提供標準化的 API、SDK 與協議自動檢測、蒐集、導出遙測數據資料 (Metrics、Log、Trace)，並支援 W3C 定義的 Http trace-context 規範